Erlind Sejdiu
106357

Exposé: Was kostet Observability? Eine quantitative Analyse des Laufzeit- und Datenoverheads von OpenTelemetry

Seminararbeit
der Hochschule Karlsruhe
im Master Studiengang Informatik
im Wintersemester 2026/27

Betreuer: [Name der betreuenden Person]
Themengruppe: [Themengruppe des Seminars]

Inhaltsverzeichnis

0.1 Messgrößen 2
0.2 Versuchsdurchführung 2
0.3 Präsentation 2

1 Zeitplan **3**

Literaturverzeichnis **3**

Faktor	Stufen
Instrumentierungsgrad	G0: keine Instrumentierung (Baseline); G1: automatische Instrumentierung der Systemgrenzen (HTTP, Datenbank); G2: G1 zuzüglich manueller Spans für fachlich relevante Funktionen; G3: ein Span für jede Funktion
Sampling	100 %, 10 % und 1 % (Head-Sampling)
Export	aktiv (OTLP an Collector) vs. deaktiviert (Spans werden erzeugt, aber verworfen)
Last	25 %, 50 %, 75 % und 90 % des in G0 ermittelten maximalen Durchsatzes

Tabelle 1: Faktoren und Stufen der Messungen

Die Stufe „Export deaktiviert“ dient dazu, die Kosten für das Erzeugen von Spans von denen für Serialisierung und Export zu trennen (H4). Um die Anzahl der Messungen beherrschbar zu halten, werden nicht alle Kombinationen vollständig vermessen: Der Mikrobenchmark variiert Instrumentierungsgrad bzw. Aufruftiefe, Sampling und Export bei fester Last; der Makrobenchmark variiert Instrumentierungsgrad, Sampling und Last bei aktivem Export.

0.1 Messgrößen

- **Laufzeit:** Antwortzeit (Median, p95, p99), Durchsatz,
- **Ressourcen:** CPU-Auslastung und Speicherbedarf der Anwendung, bei Java zusätzlich Garbage-Collection-Aktivität,
- **Datenmenge:** Spans pro Anfrage, Bytes pro Span (OTLP), übertragenes Datenvolumen pro Sekunde, Speicherbedarf im Analyse-Backend hochgerechnet auf eine Million Anfragen,
- **Infrastruktur:** CPU- und Speicherbedarf des OpenTelemetry Collectors.

0.2 Versuchsdurchführung

Die Messungen werden auf dedizierter Hardware durchgeführt, um Störeinflüsse durch geteilte Cloud-Ressourcen zu vermeiden. Alle Komponenten werden containerisiert (Docker Compose), die Last wird reproduzierbar mit einem Lastgenerator (z. B. k6) erzeugt, und die Durchführung wird skriptgesteuert automatisiert. Gemäß den Empfehlungen von Georges et al. [1] wird jede Konfiguration nach einer Aufwärmphase mehrfach gemessen; die Ergebnisse werden mit Konfidenzintervallen angegeben. Messskripte, Konfigurationen und Rohdaten werden in einem Git-Repository veröffentlicht, sodass die Messungen nachvollziehbar und wiederholbar sind.

0.3 Präsentation

Die Ergebnisse werden in einem Abschlussvortrag vorgestellt. Kern des Vortrags sind wenige, aussagekräftige Visualisierungen, insbesondere der Overhead in Abhängigkeit von Spans pro Anfrage sowie von der Last, und daraus abgeleitete Empfehlungen für die Wahl von Instrumentierungsgrad und Sampling.

1 Zeitplan

Phase 1: Literaturrecherche und Versuchsplanung

Ziel: Aufarbeitung des Stands der Technik, Auswahl des Makrobenchmarks, Festlegung der Messgrößen.

Artefakt: Kapitel Grundlagen und Stand der Technik, Versuchsplan.

Phase 2: Aufbau der Messumgebung

Ziel: Umsetzung des Mikrobenchmarks, Inbetriebnahme des Makrobenchmarks, Automatisierung der Messungen.

Artefakt: Messumgebung im Git-Repository, Pilotmessung.

Phase 3: Durchführung der Messungen

Ziel: Vermessung aller geplanten Konfigurationen.

Artefakt: Rohdaten aller Messreihen.

Phase 4: Auswertung

Ziel: Statistische Auswertung, Überprüfung der Hypothesen, Ableitung von Empfehlungen.

Artefakt: Diagramme und Auswertungsskripte.

Phase 5: Ausarbeitung und Vortrag

Ziel: Schriftliche Ausarbeitung und Vorbereitung des Abschlussvortrags.

Artefakt: Seminararbeit und Präsentationsfolien.

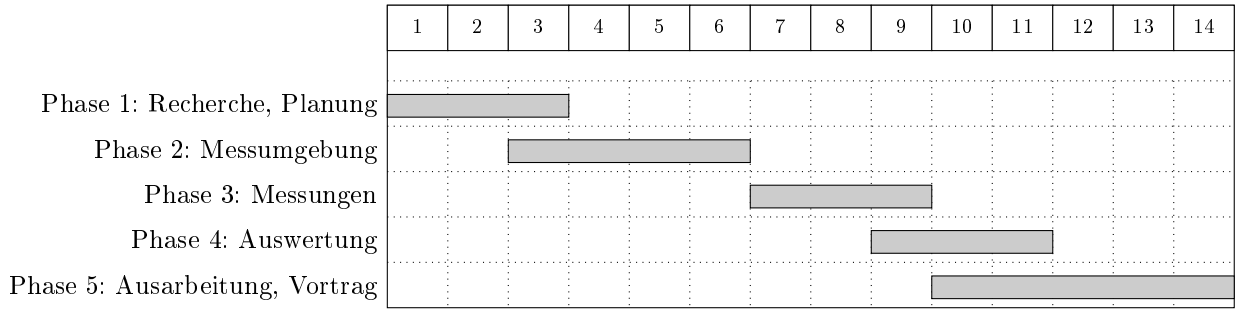


Abbildung 1: Zeitplan in Wochen [an Seminartermine anpassen]

Literaturverzeichnis

[1] A. Georges, D. Buytaert und L. Eeckhout, „Statistically Rigorous Java Performance Evaluation," in *Proceedings of the 22nd ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '07)*, 2007, S. 57–76. DOI: 10.1145/1297027.1297033